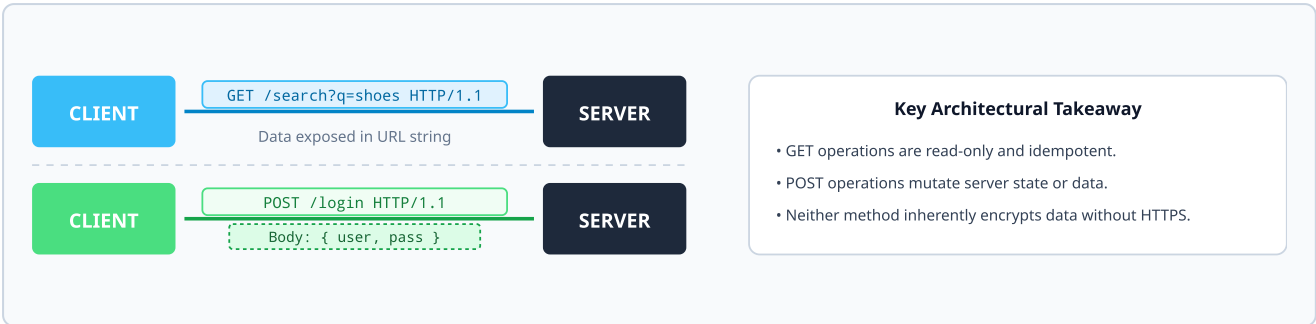


GET vs POST: HTTP Methods

Technical Reference Guide for Web Architecture, Security, and API Design

Data Transmission Architecture



Technical Comparison Matrix

Feature	GET Method	POST Method
Primary Purpose	Retrieve data from the server	Submit or modify data on the server
Data Location	Appended to URL string (?key=value)	Carried inside the HTTP request body
Data Visibility	Visible in URL bar, history, and logs	Hidden from URL bar and browser address field
Payload Capacity	Restricted by URL limits (~2,048 chars)	Unrestricted (governed by server capacity)
Caching & History	Cached by default; saved in history	Never cached; excluded from browser history
Bookmarking	Fully bookmarkable with parameters	Cannot be bookmarked with input state
Idempotency	Idempotent: Multiple calls return same state	Non-idempotent: Re-calls create side effects
Page Refresh	Refreshes safely without warnings	Triggers "Confirm Form Resubmission" alert

Implementation Guidelines

When to Use GET

- **Search & Filtering:** Passing query parameters where shareable links are beneficial (e.g., `/products?cat=shoes`).

When to Use POST

- **Sensitive Operations:** Transmitting logins, passwords, or personal data to keep them out of server logs.

Extended HTTP Methods in REST Architecture

While GET and POST handle core web navigation and form submission, modern REST APIs utilize additional standard HTTP verbs to express specific CRUD operations cleanly.

Method	CRUD Action	Safe?	Idempotent?	Typical Usage
GET	Read	Yes	Yes	Retrieve resource representation
POST	Create	No	No	Create new subordinate resource
PUT	Replace / Update	No	Yes	Replace entire resource or create if absent
PATCH	Partial Update	No	No	Apply partial modifications to resource
DELETE	Delete	No	Yes	Remove designated resource
HEAD / OPTIONS	Metadata / CORS	Yes	Yes	Fetch headers / Check allowed methods

Code Implementation Patterns

HTML Form (GET vs POST)

```
<form action="/search" method="GET">
  <input type="text" name="q" placeholder="Search...">
  <button type="submit">Search</button>
</form>

<form action="/login" method="POST">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="submit">Log In</button>
</form>
```

JavaScript Fetch API

```
// GET Request with Query Params
const res = await fetch('/api/items?cat=tech');
const data = await res.json();

// POST Request with JSON Payload
const response = await fetch('/api/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Alice', role: 'Dev' })
});
```

Security & Best Practices

CSRF Vulnerabilities

State-changing POST requests are vulnerable to Cross-Site Request Forgery. Always enforce Anti-CSRF tokens or SameSite=Strict/Lax cookie attributes.

Idempotency Keys

For critical POST operations (e.g., payment processing), pass an Idempotency-Key header to prevent double-charging on network retries.

